HOBBLY

Transaction State Machine & Services Platform

Deterministic FSM, Idempotent Orchestration & Escrow Economics for Commerce

Igor Grybek

Author & Quantitative Modeler | Algorithmic Reference Architecture & Decision Model

Live Case Study | GitHub Repository

Methodological Alignment: PSD2 Idempotency Principles | Escrow FSM Design

Executive Summary & Strategic Context

In two-sided booking marketplaces and service-commerce platforms, transactions represent extended, asynchronous operational commitments rather than instant retail purchases. Platforms frequently display rapid Gross Merchandise Value (GMV) growth while suffering severe operating cash burn due to hidden transaction friction: inventory race conditions, double charging under network latency, unhandled merchant cancellations, and manual customer support escalations.

Hobbly is an open-source transaction state machine prototype authored by **Igor Grybek** to enforce transactional rigor in asynchronous commerce workflows. Grounded in corporate banking cash-monitoring controls and SGH financial systems modeling, Hobbly operates under the defining principle: *exceptions determine the operating model*. Implemented as a deterministic Python/SQLite finite state machine (73 LOC), Hobbly models atomic booking holds, SHA-256 idempotency key deduplication, and automated ledger reconciliation across cancellations and refunds under synthetic unit-economics constraints.

System Architecture & State Machine Mechanics

Hobbly isolates transaction states, payment tokens, and operational workflows into four modular layers:

1. **Atomic Inventory Reservation Core:** Manages listing capacity via transactional database locks and expiration countdowns, preventing double-booking and overselling in simulated concurrent booking flows.
2. **Cryptographic Idempotency Filter:** Every client payment request requires a unique `Idempotency-Key`. Repeated network attempts return cached transaction results without triggering duplicate charges. Reusing an idempotency key across differing booking IDs is immediately blocked with a `409 CONFLICT` security fault.
3. **Deterministic Finite State Automaton (FSM):** Governs transitions across eight explicit states: `PUBLISHED`, `RESERVED`, `PENDING_PAYMENT`, `CONFIRMED`, `PAYMENT_FAILED`, `CANCELLED`, `REFUNDED`, `COMPLETED`. Invalid transitions are strictly rejected.
4. **Dual-Entry Escrow Reconciliation Ledger:** Maintains real-time balance integrity across gross captured funds, payment processor fees, merchant escrow, and automated refund disbursements. Guarantees zero unallocated financial drift.

5-Dimensional Multidimensional Operating Model

In accordance with the 5-Dimensional Multidimensional Operating Model, Hobbly integrates five operational dimensions:

- **Technical Architecture:** Python finite state engine with SQLite transaction persistence, atomic ACID database transactions, and OpenAPI 3.1 REST contracts.
- **Operating Workflow (RACI):** Activity Organizer (publishes capacity and confirms fulfillment) → End Customer (initiates payment and executes self-service cancellations within policy) → Operations Specialist (monitors exception queues and arbitrates disputes) → Finance Manager (authorizes batch disbursements).
- **Financial Model & Unit Margin Breakeven:** Analyzes unit economics on a modeled baseline €50.00 booking at a 10% take-rate (€5.00 revenue). Fixed gateway processing (€1.00) and expected fraud loss (€0.50) leave €3.50 in gross margin. To secure profitability, customer support friction must remain strictly below €3.50 per transaction, illustrating how automated self-service workflows model a theoretical 40.0% net contribution margin (+€2.00/booking) in unit scenarios.
- **Risk & Governance Model:** Demonstrates architectural separation of booking state from payment tokenization, modeling idempotency and audit controls that align with PSD2 SCA principles and consumer refund mandates.
- **Scaling & Integration:** Conceptual architecture designed for distributed lock management and automated escrow milestone releases.

Benchmark Performance (Synthetic Unit Test Fixture)

The reference implementation is verified via 8 automated regression test suites on synthetic scenarios in the public repository (hobbly-booking-payments on GitHub):

Benchmark Metric	Test Run Score	Reference Baseline	Operational Significance
Reconciliation Balance Leakage	**€0.00**	Variable / Drift	Net zero balance across simulated capture and full-refund flows.
Duplicate Charge Protection	**100.0%**	1.5–3.0% Error	Replay requests return cached receipts; zero duplicate debits in test suite.
Cross-Booking Collision Block	**100.0%**	Vulnerable	Conflicting keys rejected immediately with explicit exception audit.
Modeled Support Headroom	**€2.00** / **tx**	-€0.50 / tx	Modeled margin headroom under synthetic unit-economic assumptions.
Modeled Take-Rate Retention	**40.0%**	-10.0% to 15.0%	Modeled net contribution across completed synthetic transaction flows.

Architecture Decision Records (ADRs) & API Contracts

All architectural trade-offs are formally governed by RFC-style Architecture Decision Records:

- **ADR-0001:** Deterministic Finite State Automaton (FSM) for Booking Exception Ownership.
- **ADR-0002:** Cryptographic SHA-256 Idempotency Keys for Duplicate Payment Elimination.
- **ADR-0003:** Dual-Entry Escrow Reconciliation & Net Contribution Economics Guardrails.
- **OpenAPI 3.1 Contract:** Endpoints for ‘/v1/listings/id/reserve‘, ‘/v1/payments/capture‘, and ‘/v1/bookings/id/cancel‘.

Methodological & Confidentiality Notice: This architectural whitepaper utilizes validated synthetic benchmark datasets and open institutional commerce frameworks (PCI-DSS, PSD2). No confidential corporate banking or private client data from active or past institutional engagements (BNP Paribas CIB, Santander CIB) is contained herein. © 2026 Igor Grybek. All rights reserved.